

Projet DevSecOps

Etude Technique des Sources de Données et Modes de Collecte d'Evenements (Plateforme IntelliOps)

Architecture d'Ingestion - Sources de Données et Événements

Technical Study of Data Sources & Event Collection Modes

Auteur : LACHHAB Mohammed

Filière : GINF2

Etablissement : ENSA de Tanger

Année universitaire : 2025-2026

Rapport dédié exclusivement à l'étude technique des sources de données

et des modes de collecte d'événements de la plateforme IntelliOps.

Table des matières

1 Synthèse Comparative des Modes de Collecte **2**
1.1 Mode Hybride Push + Pull (Webhook + REST API) 2
1.2 Mode Direct Push (Single-Step) 2
1.3 Tableau Récapitulatif par Technologie 3

2 Etude Détaillée par Source de Données **3**
2.1 GitHub Actions 3
2.2 GitLab CI/CD 4
2.3 Jenkins 5
2.4 SonarQube Server et SonarCloud 6
2.5 OWASP ZAP 8
2.6 Trivy 9
2.7 Prometheus et Alertmanager 10
2.8 Docker et Logs Linux 11
2.9 Kubernetes 12

1 Synthèse Comparative des Modes de Collecte

La plateforme IntelliOps repose sur une architecture d'ingestion qui centralise les événements de sécurité et de qualité provenant de sources hétérogènes (GitHub, GitLab CI/CD, Jenkins, SonarQube/SonarCloud, OWASP ZAP, Trivy, Prometheus/Alertmanager, Docker & Logs Linux, Kubernetes). Avant tout transfert vers l'Agent 1, les événements bruts transitent par un point d'entrée unique :

`POST /internal/agent1/raw-events`

Deux paradigmes d'ingestion coexistent, choisis selon la nature de chaque source.

1.1 Mode Hybride Push + Pull (Webhook + REST API)

Ce mode s'applique aux plateformes SaaS ou persistantes qui n'envoient, via leur webhook, que des *métadonnées* (statut, identifiant de build, clé de projet) et non le contenu détaillé des logs ou des vulnérabilités.

Déroulement :

1. La source émet un *webhook* (Push) vers un endpoint dédié, par exemple `POST /v1/ingest/github`.
2. Le service d'ingestion vérifie l'authenticité de la requête (signature HMAC ou token d'en-tête) et répond immédiatement *HTTP 200 OK* pour acquitter l'événement sans bloquer l'appelant.
3. Le service déclenche ensuite un *pull* asynchrone via l'API REST de la source (par exemple `GET /repos/owner/repo/actions/jobs/job_id/logs`) pour récupérer le détail (logs de build, issues, traces).
4. Les données enrichies sont alors transmises à l'Agent 1 via `POST /internal/agent1/raw-events`.

Ce mode est retenu pour **GitHub Actions**, **GitLab CI/CD** et **SonarQube/SonarCloud**.

1.2 Mode Direct Push (Single-Step)

Ce mode s'applique aux sources capables d'envoyer, en une seule requête, l'intégralité des métadonnées et des logs/rapports : étapes CI/CD, Alertmanager, ou agents de bord (Edge / in-cluster).

Déroulement :

1. Un événement est détecté localement (échec de build, alerte, événement Kubernetes, log critique).
2. Le client (script CI, agent Vector/FluentBit/k8s-event-exporter) constitue une requête unique contenant métadonnées et payload brut (*raw_ logs*, *zap_ report.json*, rapport Trivy, etc.).
3. Le client effectue un *POST* direct vers l'endpoint d'ingestion correspondant, authentifié par un token interne (`X-IntelliOps-Token` ou Bearer token).
4. Le service valide et transmet immédiatement à l'Agent 1.

Ce mode est retenu pour **Jenkins**, **OWASP ZAP** (option conteneur éphémère), **Trivy**, **Prometheus/Alertmanager**, **Docker & Logs Linux** et **Kubernetes**. Il évite d'exposer les serveurs internes à Internet et évite de stocker leurs credentials côté plateforme.

1.3 Tableau Récapitulatif par Technologie

Outil	Domaine	Mode de déclenchement	Action de collecte	Authentification
GitHub Actions	CI/CD	Webhook (push) + API (pull)	Logs de jobs/workflows & métadonnées	Fine-grained PAT (HMAC `x-hub-signature-256`)
GitLab CI/CD	CI/CD	Webhook (push) + API (pull)	Traces de jobs & métadonnées	PAT (`PRIVATE-TOKEN`, `X-Gitlab-Token`)
Jenkins	CI/CD	Direct push (post { failure })	Tail des logs console + métadonnées	`X-IntelliOps-Token`
SonarQube / SonarCloud	Qualité code	Webhook (push) + API Issues (pull)	Vulnérabilités, code smells, bugs	Bearer User Token + HMAC webhook
OWASP ZAP	Sécurité applicative	Direct push (conteneur éphémère) ou pull (daemon)	Rapports ZAP JSON (`zap_report.json`)	`X-IntelliOps-Token` / API Key ZAP
Trivy	Sécurité des images & IaC	Direct push (CLI / Operator)	Vulnérabilités HIGH/CRITICAL	`X-IntelliOps-Token`
Prometheus / Alertmanager	Métriques & alerting	Webhook direct (pull PromQL optionnel)	Alertes groupées & dédoublonnées	Bearer Project Token
Docker & Logs Linux	Logs hôtes & conteneurs	Edge shipper (Vector / FluentBit)	Événements Docker, logs stdout/stderr, syslog, journald	`POST /v1/ingest/logs`
Kubernetes	Orchestration	In-cluster (event-exporter + DaemonSet)	Événements K8s Warning + logs pods	ServiceAccount + RBAC

TABLE 1 – Synthèse des modes de collecte par source de données.

2 Etude Détaillée par Source de Données

2.1 GitHub Actions

Déclenchement et collecte. GitHub Actions utilise un *mode hybride Push + Pull*. Le webhook GitHub (`push`, `pull_request`, `workflow_run`, `check_run`, etc.) ne transporte que des métadonnées (nom du workflow, statut, identifiant du job, SHA du commit). Dès réception du `POST /v1/ingest/github`, le service vérifie la signature HMAC `x-hub-signature-256` calculée avec le *webhook secret*, répond immédiatement *HTTP 200 OK*, puis effectue un *pull* asynchrone via l'API GitHub pour récupérer le détail des logs :

Listing 1 – Appel API REST GitHub pour récupérer les logs d'un job

```
GET /repos/{owner}/{repo}/actions/jobs/{job_id}/logs
Authorization: Bearer <PAT Fine-grained>
```

Authentification et base de données. L'authentification repose sur un *Fine-grained PAT* avec le scope `Actions: Read-only`. Ce PAT est stocké chiffré en base, associé au mapping dépôt/platforme, ainsi que le *webhook secret* utilisé pour vérifier l'integrité des requêtes entrantes.

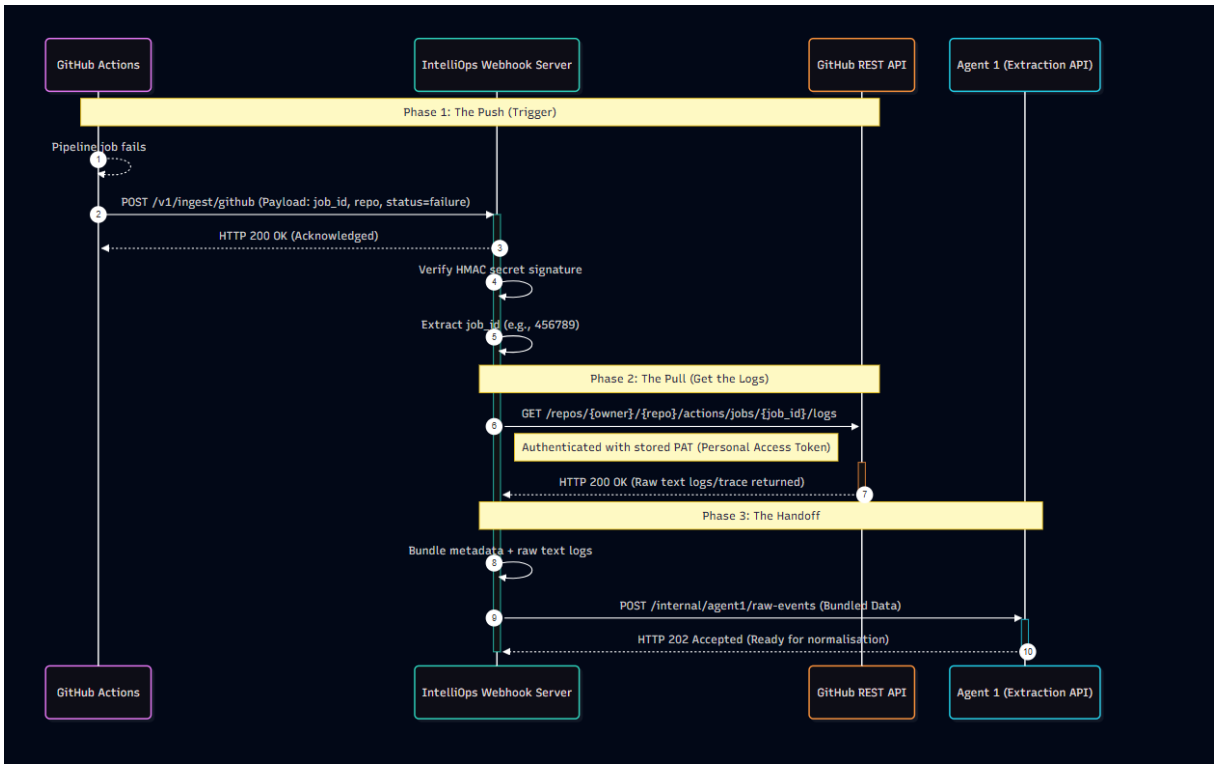


FIGURE 1 – Séquence de collecte GitHub Actions.

2.2 GitLab CI/CD

Déclenchement et collecte. Comme GitHub, GitLab CI/CD utilise un *mode hybride Push + Pull*. Le webhook GitLab (`push`, `merge request`, `pipeline`, `job`) ne transmet que des métadonnées. Le service écoute sur `POST /v1/ingest/gitlab` et vérifie l'en-tête `X-Gitlab-Token` contre le token configuré côté IntelliOps, puis répond *HTTP 200 OK*. Un pull asynchrone récupère ensuite la trace complète du job :

Listing 2 – Appel API REST GitLab pour récupérer la trace d'un job

```
GET /api/v4/projects/{project_id}/jobs/{job_id}/trace
PRIVATE-TOKEN: <PAT>
```

Authentification et base de données. L'authentification utilise un *PAT* passé en en-tête `PRIVATE-TOKEN` avec le scope `read_api`. GitLab SaaS et GitLab Self-Managed sont supportés via une *base URL* dynamique stockée en base. Les champs persistés sont : PAT chiffré, `PRIVATE-TOKEN`, `X-Gitlab-Token`, URL de base, mapping projet, et secret de webhook.

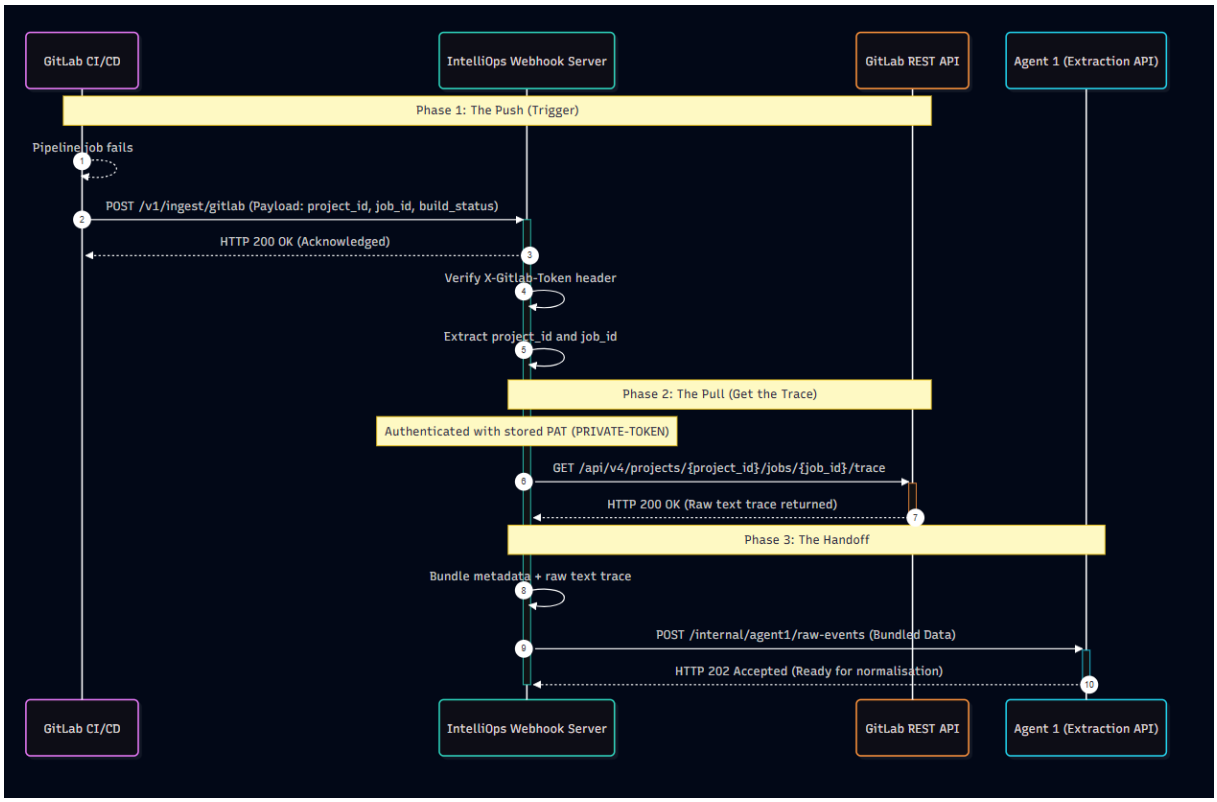


FIGURE 2 – Séquence de collecte GitLab CI/CD.

2.3 Jenkins

Pourquoi un Direct Push en une étape. Jenkins est typiquement déployé en interne et n'est pas exposé sur Internet. Un mode hybride webhook + pull exigerait soit d'exposer Jenkins aux appels entrants, soit de stocker les credentials Jenkins (utilisateur/mot de passe ou token API) côté plateforme pour poller l'API Jenkins. Le *Direct Push* en une étape élimine ces deux contraintes : le serveur Jenkins initie lui-même la requête sortante, aucune credential Jenkins n'est stockée par IntelliOps, et le périmètre d'exposition réseau est nul.

Fonctionnement. Dans le `Jenkinsfile`, le bloc `post { failure }` intercepte l'échec d'un build. Il extrait la *queue* de la console via `currentBuild.rawBuild.getLog(150)` (les 150 dernières lignes) afin d'éviter des payloads trop volumineux susceptibles de provoquer des timeouts. Les métadonnées (nom du job, numéro de build, statut, URL) et les `raw_logs` sont envoyés dans une seule requête `POST /v1/ingest/jenkins`, authentifiée par l'en-tête `X-IntelliOps-Token`.

Listing 3 – Jenkinsfile - bloc post d'envoi direct vers IntelliOps

```
pipeline {
    agent any
    stages {
        stage('Build') { steps { sh 'make build' } }
        stage('Test')  { steps { sh 'make test' } }
    }
    post {
```

```
        failure {
            script {
                def logs = currentBuild.rawBuild.getLog(150)
                def payload = [
                    job: env.JOB_NAME,
                    build: env.BUILD_NUMBER,
                    status: 'FAILURE',
                    url: env.BUILD_URL,
                    raw_logs: logs
                ]
                def json = groovy.json.JsonOutput.toJson(payload)
                sh "curl -X POST https://intelliops.local/v1/ingress/jenkins "
    +
                   "-H 'X-IntelliOps-Token: \$INTELLIOPS_TOKEN' " +
                   "-H 'Content-Type: application/json' " +
                   "-d '${json}'"
            }
        }
    }
}
```

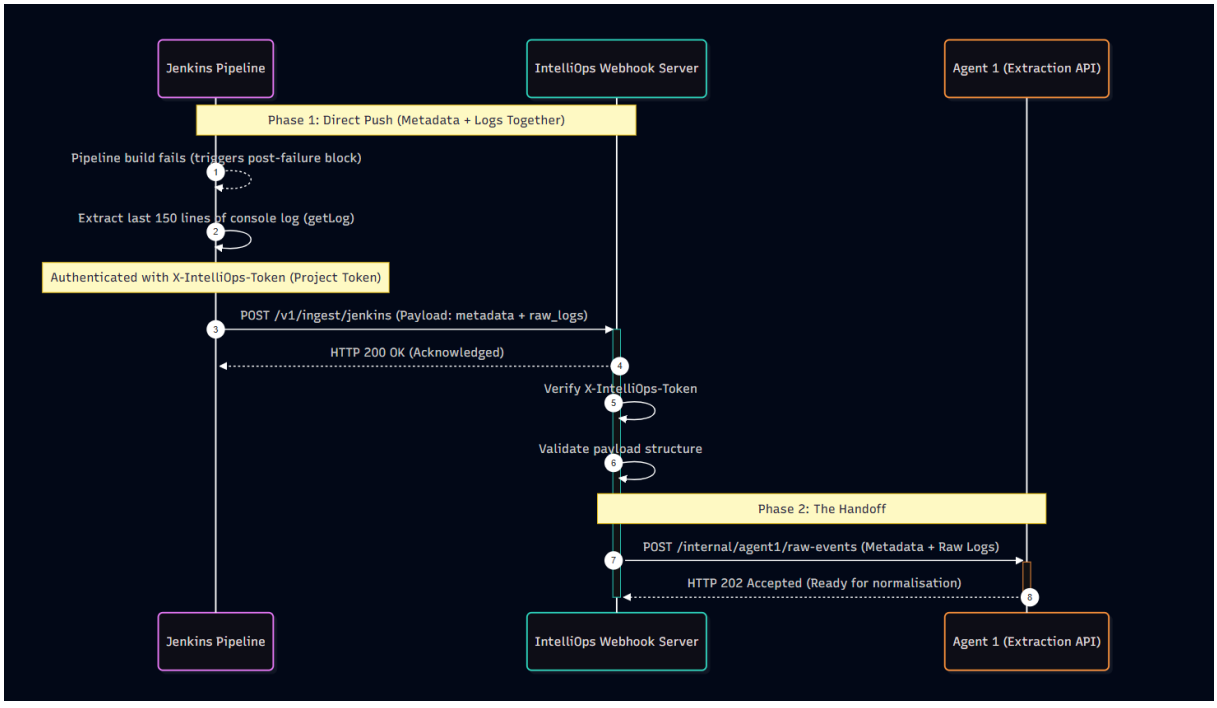


FIGURE 3 – Séquence de collecte Jenkins.

2.4 SonarQube Server et SonarCloud

Pourquoi un Push + Pull en deux étapes. Le webhook SonarQube ne transporte que le statut du *Quality Gate* (pass/fail) et des métadonnées de projet ; il ne contient ni les vulnérabilités, ni les code smells, ni les bugs avec leur fichier et leur ligne. Une collecte en deux étapes est donc indispensable : le webhook signale l'événement (Phase 1), puis une requête API récupère les détails des issues (Phase 2).

Phase 1 - Webhook du Quality Gate. Lorsque le statut du Quality Gate passe à `ERROR`, SonarQube/SonorCloud POSTe un webhook signé `X-Sonar-Webhook-HMAC-SHA256`. Le service vérifie la signature et répond *HTTP 200 OK*.

Phase 2 - Pull des issues via API REST. Le service appelle ensuite l'endpoint de recherche d'issues non résolues :

Listing 4 – Appel API SonarQube/SonorCloud pour récupérer les issues

```
GET {base_url}/api/issues/search?componentKeys={projectKey}&resolved=false
Authorization: Bearer <User Token>
```

Option 1 : SonarCloud (SaaS). `base_url` = `https://sonarcloud.io`. Pas d'hébergement à gérer, authentification par Bearer User Token SonarCloud.

Option 2 : SonarQube Self-Hosted. `base_url` est l'URL de l'instance propre à l'organisation (ex. `https://sonar.internal.company`). Le même handler unifié est utilisé, le `base_url` étant dynamique selon l'édition déclarée en base.

Champ	Description
`edition_type`	`SAAS` (SonarCloud) ou `SELF_HOSTED` (SonarQube)
`base_url`	URL de base de l'instance (`https://sonarcloud.io` ou URL propre)
`project_key`	Clé du projet Sonar (`componentKeys`)
`auth_token`	Bearer User Token (chiffré) pour l'API `/api/issues/search`
`webhook_secret`	Secret utilisé pour vérifier `X-Sonar-Webhook-HMAC-SHA256`

TABLE 2 – Champs unifiés de la base de données pour SonarQube / SonarCloud.

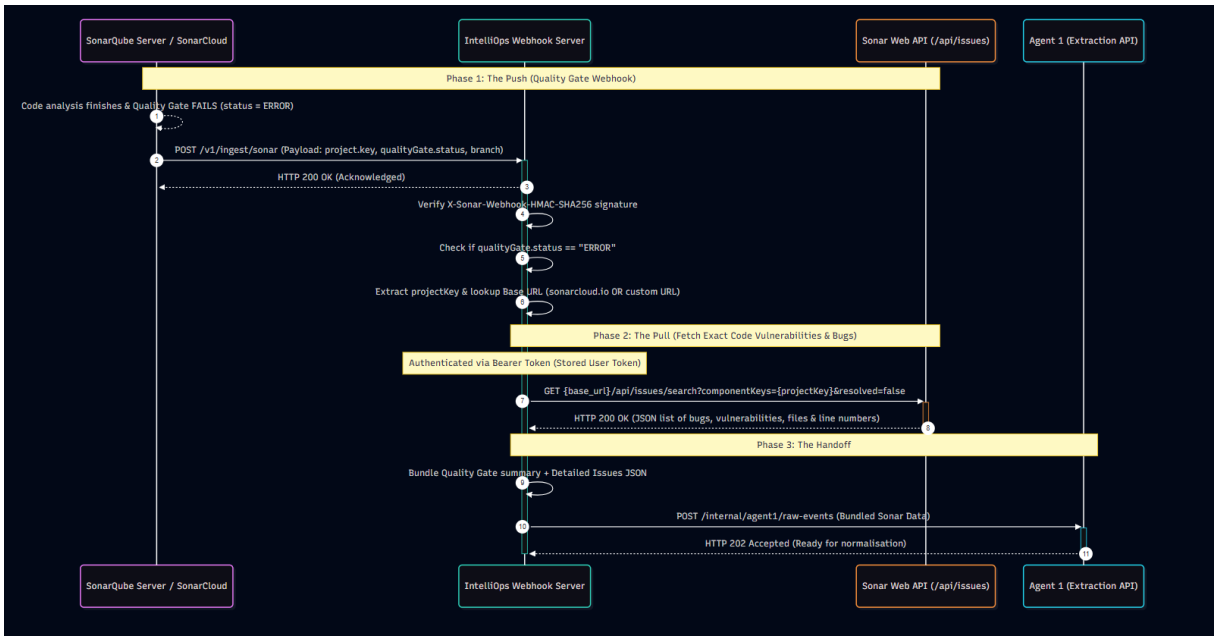


FIGURE 4 – Séquence de collecte SonarQube / SonarCloud.

Champs de la base de données (unifiés).

2.5 OWASP ZAP

Les trois cas de scan supportés.

- **Cas A** (`zap-baseline.py`) : scan de base d'une application web cible, sans fichier de définition d'API.
- **Cas B** (`zap-api-scan.py`) : scan d'API à partir d'une définition OpenAPI / Swagger / GraphQL.
- **Cas C** (`zap-full-scan.py`) : scan complet (spider + active scan) d'une application.

Deux architectures de collecte.

- **Option 1 - Conteneur CI/CD éphémère (Direct Push).** Le conteneur ZAP exécute le scan, produit `zap_report.json`, puis l'envoie en une seule requête `POST` authentifiée par `X-IntelliOps-Token` avec l'option `-I` (rapport JSON).
- **Option 2 - Daemon ZAP persistant (Pull).** Une instance ZAP persistante expose son API locale ; le service d'ingestion interroge l'API ZAP pour récupérer le rapport.

Filtrage du bruit. Avant la remise à l'Agent 1, les alertes de niveau `riskcode "0"` (Informational) sont filtrées pour ne conserver que les incidents pertinents.

Listing 5 – Scan ZAP baseline en CI et envoi direct

```
docker run -t owasp/zap2docker-stable zap-baseline.py \
    -t https://app.example.com -I -z "-config.api.disablekey=true" \
    -w /zap/wrk/
cat /zap/wrk/zap_report.json | curl -X POST https://intelliops.local/v1/ingest
   /zap \
```

```
    -H 'X-IntelliOps-Token: $INTELLIOPS_TOKEN' \
    -H 'Content-Type: application/json' --data-binary @-
```

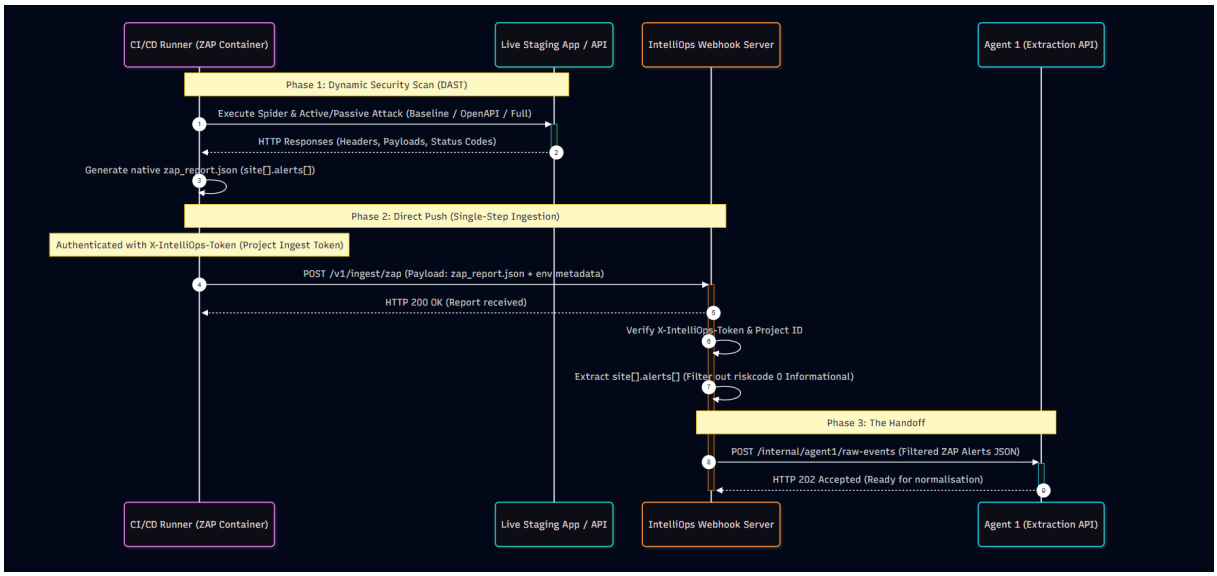


FIGURE 5 – Séquence de collecte OWASP ZAP.

2.6 Trivy

Les trois cas de scan.

- **Cas A** (`trivy image`) : analyse d'images conteneurs pour les vulnérabilités OS et dépendances applicatives.
- **Cas B** (`trivy fs`) : analyse du système de fichiers et des fichiers de verrouillage (lockfiles) des dépendances.
- **Cas C** (`trivy config` / `trivy k8s`) : analyse de configuration des manifests Terraform, Ansible et Kubernetes.

Environnements d'exécution. Trivy s'exécute soit en CLI éphémère dans le pipeline CI/CD, soit via l'opérateur Trivy dans Kubernetes.

Flux Direct Push. Le scan génère un rapport JSON qui est filtré pour ne conserver que les sévérités `HIGH` et `CRITICAL`, puis envoyé en une seule requête `POST /v1/ingest/trivy` authentifiée par `X-IntelliOps-Token`.

Listing 6 – Scan Trivy image et envoi direct des vulnérabilités critiques

```
trivy image --format json --severity HIGH,CRITICAL \
    --output trivy_report.json registry.example.com/app:latest
curl -X POST https://intelliops.local/v1/ingest/trivy \
    -H 'X-IntelliOps-Token: $INTELLIOPS_TOKEN' \
    -H 'Content-Type: application/json' --data-binary @trivy_report.json
```

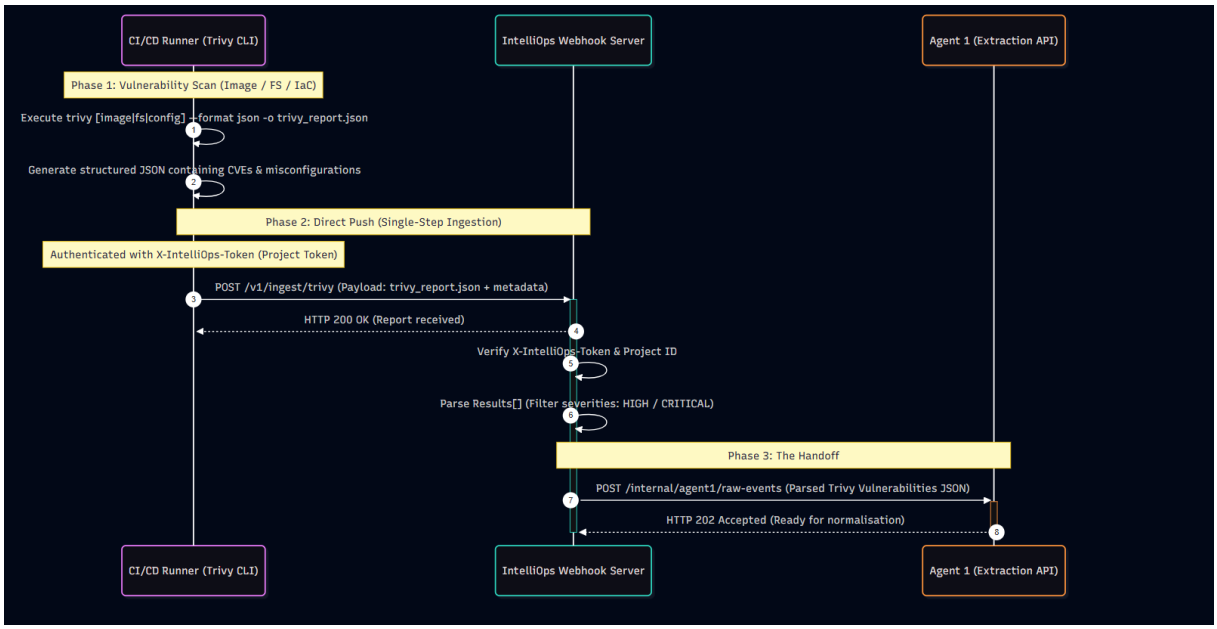


FIGURE 6 – Séquence de collecte Trivy.

2.7 Prometheus et Alertmanager

Pas de flux continu de séries temporelles. Les séries temporelles brutes (métriques toutes les 15 s) ne sont pas streamées : le volume serait trop important et l'information de sécurité n'est pas dans la métrique brute mais dans l' *alerte* qui en découle.

Les trois cas de métriques surveillées.

- **Cas A** : hôte / VM via *Node Exporter* (CPU, mémoire, disque, réseau).
- **Cas B** : conteneurs / Kubernetes via *cAdvisor* et *kube-state-metrics*.
- **Cas C** : applications / APIs via leurs exporters dédiés.

Évaluation PromQL et Alertmanager. Prometheus évalue les règles PromQL et génère des alertes. Alertmanager assure le *grouping* et la *déduplication* (`group_by`) avant de notifier IntelliOps.

Collecte. Une *Single-Step Direct Webhook Push* est utilisée : Alertmanager POSTe l'alerte groupée vers `POST /v1/ingest/prometheus`, authentifiée par un *Bearer Project Token*. Un *pull PromQL optionnel* permet d'enrichir l'alerte avec le contexte des métriques au moment de l'incident.

Listing 7 – alertmanager.yml - envoi direct vers IntelliOps

```
route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 30s
  group_interval: 5m
  receiver: 'intelliops'
receivers:
  - name: 'intelliops'
    webhook_configs:
      - url: 'https://intelliops.local/v1/ingest/prometheus'
```

```
        http_config:
          bearer_token: <PROJECT_TOKEN>
```

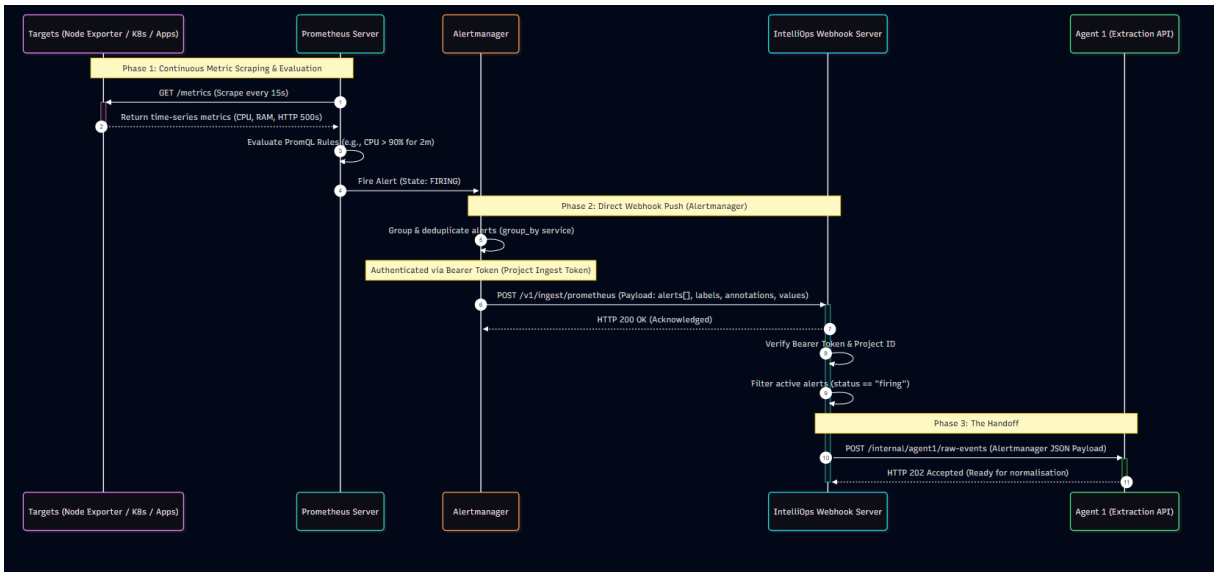


FIGURE 7 – Séquence de collecte Prometheus / Alertmanager.

2.8 Docker et Logs Linux

Les trois cas cibles sur l'hôte.

- **Cas A** : événements Docker Engine via `/var/run/docker.sock` (`oom` / `OOMKilled`, `die`, boucles de redémarrage).
- **Cas B** : logs `stdout`/`stderr` des conteneurs.
- **Cas C** : logs Linux système (`/var/log/syslog`, `/var/log/auth.log`, `journald`).

Filtrage en bordure (Edge). Un shipper léger (**Vector** ou **FluentBit**) tourne sur l'hôte, supprime localement le bruit (`INFO`/`DEBUG`) et ne pousse que les incidents critiques vers `POST /v1/ingest/logs`.

Listing 8 – vector.yaml - collecte logs et événements Docker, filtrage bruit

```
sources:
  docker_events:
    type: docker_socket
    endpoint: /var/run/docker.sock
  container_logs:
    type: file
    include: ['/var/log/containers/*.log']
  syslog:
    type: file
    include: ['/var/log/syslog', '/var/log/auth.log']

transforms:
  filter_noise:
    type: filter
    condition: |
```

```
      .level != "INFO" and .level != "DEBUG"

sinks:
  intelliops:
    type: http
    inputs: ["docker_events", "container_logs", "syslog", "filter_noise"]
    endpoint: https://intelliops.local/v1/ingest/logs
    headers:
      X-IntelliOps-Token: "${INTELLIOPS_TOKEN}"
    encoding: json
```

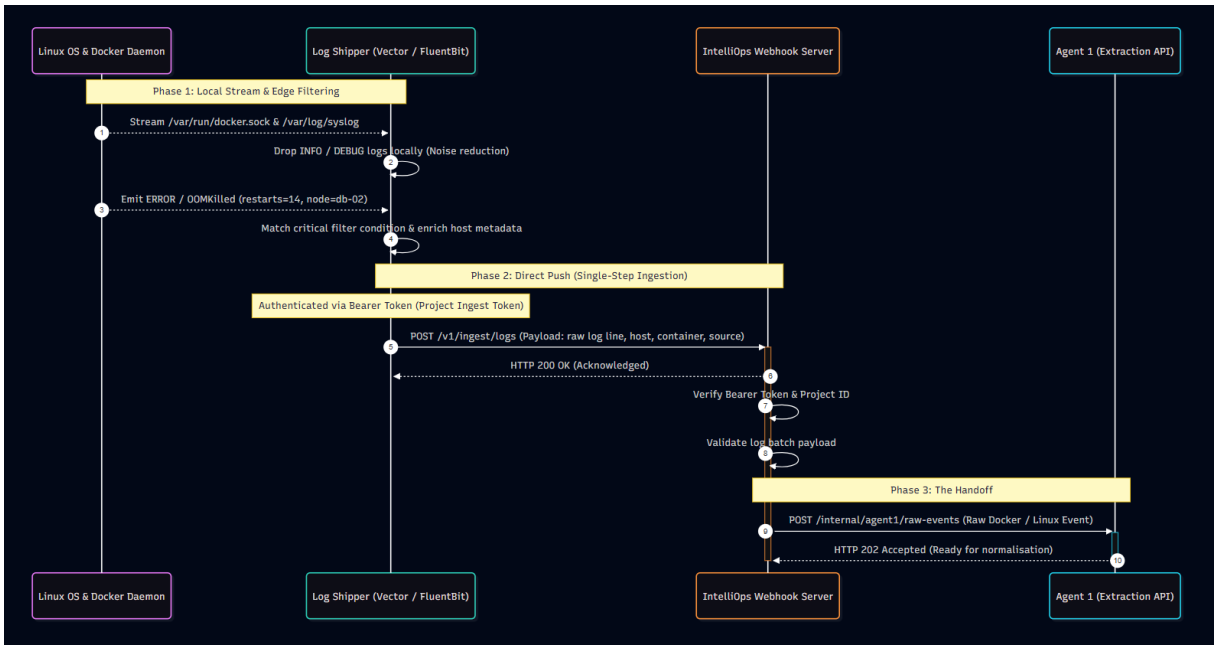


FIGURE 8 – Séquence de collecte Docker et logs Linux.

2.9 Kubernetes

Les trois cas cibles.

- **Cas A** : échecs du cycle de vie Pod/Conteneur via l'API Événements Kubernetes (`CrashLoopBackOff`, `ImagePullBackOff`, `OOMKilled`, `FailedScheduling`).
- **Cas B** : alertes de santé des nodes (`NodeNotReady`, `DiskPressure`, `MemoryPressure`).
- **Cas C** : logs des pods de microservice en échec.

Architecture in-cluster. Un `kubernetes-event-exporter` (*Déploiement*) capture les événements du cluster, couplé à un *DaemonSet* Vector/FluentBit pour les logs des pods. L'authentification s'appuie sur un *ServiceAccount* Kubernetes avec les *RBAC* nécessaires (lecture des `events`, `pods`, `nodes`).

Filtrage. Les événements de niveau `Normal` sont filtrés localement ; seuls les événements `Warning` sont poussés vers `POST /v1/ingest/k8s`.

Listing 9 – ConfigMap - filtrage et routage des événements Kubernetes vers IntelliOps

```
apiVersion: v1
```

```
kind: ConfigMap
metadata:
  name: intelliops-k8s-collector
  namespace: intelliops
data:
  config.yaml: |
    sources:
      k8s_events:
        type: kubernetes_events
    transforms:
      keep_warnings:
        type: filter
        condition: | .reason != "Normal"
    sinks:
      intelliops:
        type: http
        inputs: ["k8s_events", "keep_warnings"]
        endpoint: https://intelliops.local/v1/ingest/k8s
        headers:
          X-IntelliOps-Token: "${INTELLIOPS_TOKEN}"
        encoding: json
```

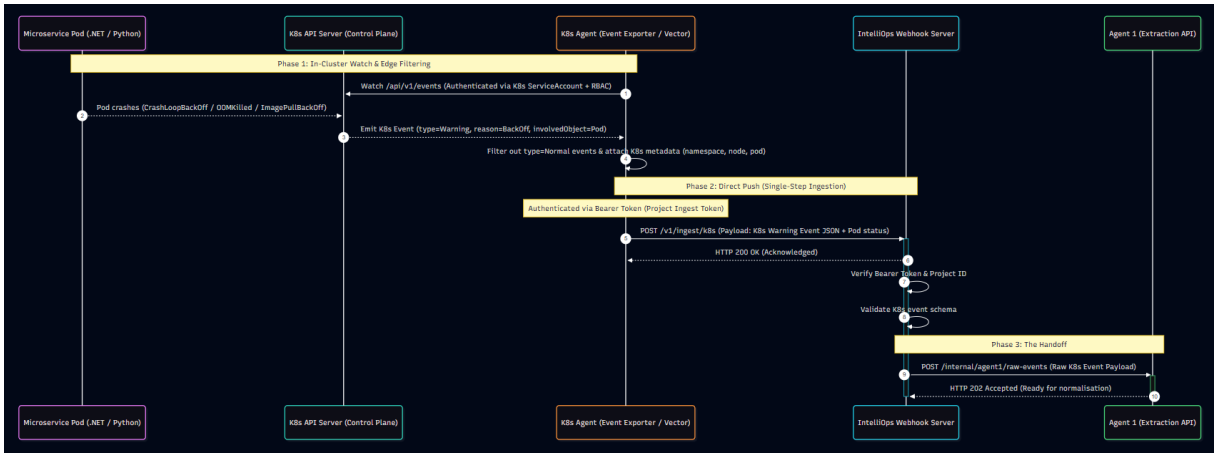


FIGURE 9 – Séquence de collecte Kubernetes.